

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

_____ * _____



BÁO CÁO CUỐI KÌ 2024.2

HỆ THỐNG QUẢN LÝ CỬA HÀNG TRANG SỨC

Học phần: Thực hành Cơ sở dữ liệu

Mã học phần: IT3290

Mã lớp: 156783

Giảng viên: Vũ Tuyết Trinh

Nhóm 11:

Quách Nguyễn Quỳnh Anh 20236020

Nguyễn Nhật Linh 20236040

Đinh Xuân Thanh 20236054

Hà Nội, ngày 10 tháng 6 năm 2025

MỤC LỤC

I.	Giới Thiệu Về Hệ Thống	2
A.	Bối cảnh	2
B.	Mục Tiêu	3
II.	Mô Tả Hệ Thống.....	3
A.	Quản Lý Trang Sức	4
B.	Quản Lý Khách Hàng	5
C.	Quản lý đơn hàng.....	5
D.	Báo cáo & Thống kê:.....	6
III.	Thiết kế cơ sở dữ liệu	6
A.	Bảng.....	6
1.	Bảng Users (Người dùng).....	6
2.	Bảng Customers (Khách hàng)	6
3.	Bảng Products (Sản phẩm)	7
4.	Bảng Orders (Đơn hàng)	7
5.	Bảng Order_Items (Chi tiết đơn hàng)	7
B.	Mối quan hệ giữa các thực thể	7
C.	Sơ đồ thực thể ERD.....	8
D.	Sơ đồ quan hệ	8
IV.	Tạo bảng cho cơ sở dữ liệu	9
A.	Bảng người dùng (Users)	9
B.	Bảng khách hàng (Customers)	9
C.	Bảng sản phẩm trang sức (Products).....	10
D.	Bảng đơn hàng (Orders).....	10
E.	Bảng chi tiết đơn hàng (Order_Items).....	11
V.	Các chức năng nghiệp vụ.....	11
A.	Quản lý sản phẩm	11
B.	Quản lý đơn hàng.....	18
C.	Quản lý khách hàng	22

I. Giới Thiệu Về Hệ Thống

A. Bối cảnh

- Trong ngành kinh doanh trang sức, việc quản lý sản phẩm, khách hàng và đơn hàng là rất quan trọng.

- Hệ thống giúp tối ưu hóa hoạt động của cửa hàng trang sức cá nhân, đảm bảo quản lý hàng hóa chính xác, nâng cao trải nghiệm khách hàng và tối ưu doanh thu, hệ thống không chỉ hỗ trợ quá trình bán hàng mà còn giúp cửa hàng cải thiện chất lượng dịch vụ, theo dõi xu hướng mua sắm và đưa ra các quyết định kinh doanh dựa trên dữ liệu thực tế..

B. Mục Tiêu

- Tự động hóa quá trình quản lý sản phẩm, khách hàng và đơn hàng.
- Theo dõi chi tiết từng mẫu trang sức, đảm bảo không thất thoát tài sản.
- Cung cấp hệ thống tích lũy điểm thưởng và ưu đãi cho khách hàng trung thành.
- Đảm bảo bảo mật thông tin khách hàng.

II. Mô Tả Hệ Thống

Khi khách hàng đến cửa hàng hoặc thực hiện mua sắm trực tuyến, hệ thống hỗ trợ các bước xử lý dịch vụ như sau:

1. Tiếp nhận thông tin khách hàng

- Nhân viên thu thập thông tin cơ bản: tên, số điện thoại, email.
- Nếu khách đã từng mua hàng, hệ thống tự động tra cứu và hiển thị lịch sử giao dịch, giúp tư vấn phù hợp hơn.

2. Tư vấn và chọn sản phẩm

- Khách hàng có thể xem và lựa chọn sản phẩm theo danh mục: nhẫn, dây chuyền, bông tai, vòng tay...
- Nhân viên tư vấn theo nhu cầu về kiểu dáng, chất liệu, dịp sử dụng (cưới, quà tặng, dùng hàng ngày...).
- Hệ thống hỗ trợ tìm kiếm sản phẩm nhanh theo tên, loại, chất liệu, mức giá hoặc các bộ sưu tập đặc biệt.

3. Kiểm tra tồn kho và xác nhận sản phẩm

- Khi khách chọn sản phẩm, hệ thống tự động kiểm tra số lượng tồn kho.
- Nếu sản phẩm còn hàng, hệ thống cho phép thêm vào đơn hàng.
- Với sản phẩm sắp hết hoặc hết hàng, nhân viên có thể tư vấn đặt hàng trước hoặc liên hệ khi có lại hàng.

4. Tạo đơn hàng và thanh toán

- Sau khi xác nhận sản phẩm, nhân viên tạo đơn hàng trên hệ thống.

- Đơn hàng được thiết lập trạng thái “pending” cho đến khi hoàn tất thanh toán.
- Khách hàng thanh toán toàn bộ hoặc đặt cọc trước (tùy chính sách). Hệ thống tự động cập nhật trạng thái đơn và trừ tồn kho.

5. Giao hàng (nếu mua online)

- Nếu khách mua trực tuyến hoặc yêu cầu giao hàng, hệ thống ghi nhận địa chỉ và thời gian giao nhận.
- Đơn hàng được chuyển sang bộ phận giao hàng, trạng thái cập nhật theo tiến trình: đang xử lý, đang giao, đã giao.

6. Chăm sóc sau bán

- Sau giao dịch, hệ thống lưu thông tin đơn hàng vào hồ sơ khách hàng.
- Hệ thống tự động gợi ý sản phẩm phù hợp trong tương lai hoặc gửi ưu đãi dịp đặc biệt (sinh nhật, ngày kỷ niệm).
- Nếu khách hàng có khiếu nại, hệ thống ghi nhận và theo dõi quá trình xử lý, đảm bảo minh bạch.

Hệ thống hiện tập trung vào các chức năng cốt lõi gồm quản lý sản phẩm, khách hàng và đơn hàng. Trong các phiên bản mở rộng, hệ thống có thể bổ sung chức năng giảm giá, thanh toán, phân quyền nhân viên. Các yếu tố này sẽ giúp hệ thống trở nên linh hoạt và sát thực tế hơn trong môi trường thương mại điện tử và bán lẻ hiện đại.

A. Quản Lý Trang Sức

- Hệ thống quản lý chi tiết từng sản phẩm trang sức, bao gồm các thông tin cơ bản như mã sản phẩm, tên, loại, chất liệu, trọng lượng, số lượng tồn kho và giá bán.
- Theo dõi số lượng tồn kho liên tục, cảnh báo khi sản phẩm sắp hết hàng (dưới mức quy định) để kịp thời nhập thêm, tránh thiếu hụt hàng hóa gây gián đoạn bán hàng.
- Hỗ trợ phát hiện các sản phẩm trùng lặp dựa trên các thuộc tính chính như tên, loại, chất liệu và trọng lượng, từ đó tự động gộp tồn kho vào sản phẩm có mã nhỏ nhất và loại bỏ các bản trùng không cần thiết, giúp dữ liệu sản phẩm chính xác, sạch sẽ.
- Tăng tốc tìm kiếm sản phẩm theo tên, giúp nhân viên dễ dàng tìm kiếm và xử lý đơn hàng nhanh chóng, nâng cao hiệu suất làm việc.
- Quản lý giá sản phẩm một cách linh hoạt, có thể cập nhật giá hàng loạt theo phần trăm tăng hoặc giảm, giúp điều chỉnh giá phù hợp với thị trường nhanh chóng.

B. Quản Lý Khách Hàng

- Lưu trữ và quản lý thông tin khách hàng chi tiết, bao gồm tên, email, và các dữ liệu liên quan, giúp theo dõi lịch sử mua hàng và các giao dịch đã thực hiện.
- Phân loại khách hàng theo số lượng đơn hàng và tổng số tiền đã chi tiêu, từ đó hỗ trợ xây dựng các chương trình khách hàng thân thiết, khuyến mãi hoặc ưu đãi riêng biệt cho từng nhóm khách hàng.
- Cung cấp chức năng tìm kiếm khách hàng nhanh chóng thông qua email hoặc các thông tin cá nhân khác, giúp nhân viên dễ dàng hỗ trợ và tra cứu dữ liệu khách hàng khi cần.
- Hỗ trợ nhận diện và tìm kiếm những khách hàng lâu ngày không phát sinh giao dịch để triển khai các chiến dịch kích hoạt lại khách hàng, giữ chân khách hàng trung thành.

C. Quản lý đơn hàng

- Quản lý toàn bộ quá trình tạo, cập nhật, và xóa đơn hàng từ lúc khách hàng đặt hàng cho đến khi giao dịch hoàn tất. Khi tạo đơn hàng mới, trạng thái được tự động thiết lập là ‘pending’ nhằm chuẩn hóa quy trình xử lý đơn hàng.
- Khi thêm sản phẩm vào đơn hàng, hệ thống tự động kiểm tra tồn kho để đảm bảo số lượng sản phẩm trong kho đủ đáp ứng, tránh trường hợp bán hàng vượt quá số lượng có sẵn.
- Tự động trừ tồn kho khi đơn hàng được tạo thành công, và cộng trả lại tồn kho nếu đơn hàng bị hủy hoặc xóa, đảm bảo dữ liệu tồn kho luôn chính xác và cập nhật kịp thời.
- Hiển thị danh sách các đơn hàng mới nhất giúp nhân viên dễ dàng theo dõi, xử lý các đơn hàng đang chờ hoặc mới phát sinh.
- Cung cấp báo cáo doanh thu theo từng tháng, giúp quản lý có cái nhìn tổng quan về hiệu quả kinh doanh, nhận diện các mùa cao điểm hoặc suy giảm doanh thu để có chiến lược phù hợp.
- Lưu lại lịch sử các đơn hàng đã bị xóa, bao gồm các thông tin quan trọng như mã đơn, khách hàng, tổng tiền và trạng thái đơn, giúp đảm bảo minh bạch trong quản lý và hỗ trợ kiểm tra khi cần thiết.

D. Báo cáo & Thống kê:

- Báo cáo sản phẩm sắp hết hàng giúp kịp thời nhập bổ sung, tránh tình trạng thiếu hàng gây ảnh hưởng đến doanh số và uy tín cửa hàng.
- Hiện thị danh sách top 5 sản phẩm bán chạy nhất dựa trên tổng số lượng bán ra, hỗ trợ việc lựa chọn sản phẩm để đẩy mạnh marketing hoặc dự trữ kho.
- Thống kê top 5 khách hàng tiêu biểu dựa trên tổng số tiền đã chi tiêu và số đơn hàng, tạo cơ sở cho các chương trình ưu đãi cá nhân hóa nhằm giữ chân khách hàng trung thành.
- Thống kê doanh thu theo từng tháng giúp quản lý theo dõi xu hướng tài chính của cửa hàng, từ đó đưa ra các quyết định kinh doanh hợp lý và kịp thời.
- Cập nhật liên tục các số liệu về sản phẩm, khách hàng và đơn hàng để đảm bảo dữ liệu luôn mới nhất, hỗ trợ công tác quản lý và phân tích hiệu quả hoạt động của cửa hàng.

III. Thiết kế cơ sở dữ liệu

A. Bảng

1. Bảng Users (Người dùng)

Khóa chính: id (integer)

Thuộc tính:

username (kiểu chuỗi)

email (kiểu chuỗi)

password_hash (kiểu chuỗi)

created_at (thời gian không có múi giờ)

is_active (boolean)

2. Bảng Customers (Khách hàng)

Khóa chính: id (integer)

Thuộc tính:

name (kiểu chuỗi)

email (kiểu chuỗi)

phone (kiểu chuỗi)

address (kiểu chuỗi)

created_at (thời gian không có múi giờ)

3. Bảng Products (Sản phẩm)

Khóa chính: id (integer)

Thuộc tính:

name (kiểu chuỗi)

description (kiểu text)

price (kiểu số)

stock (kiểu integer)

category (kiểu chuỗi)

material (kiểu chuỗi)

weight (kiểu số)

image_url (kiểu chuỗi)

created_at (thời gian không có múi giờ)

4. Bảng Orders (Đơn hàng)

Khóa chính: id (integer)

Thuộc tính:

customer_id (integer) [Khóa ngoại đến Customers.id]

total_amount (kiểu số)

status (kiểu chuỗi)

created_at (thời gian không có múi giờ)

5. Bảng Order_Items (Chi tiết đơn hàng)

Khóa chính: id (integer)

Thuộc tính:

order_id (integer) [Khóa ngoại đến Orders.id]

product_id (integer) [Khóa ngoại đến Products.id]

quantity (integer)

price (numeric)

B. Môi quan hệ giữa các thực thể

- *Customers* → *Orders* (Một – nhiều)

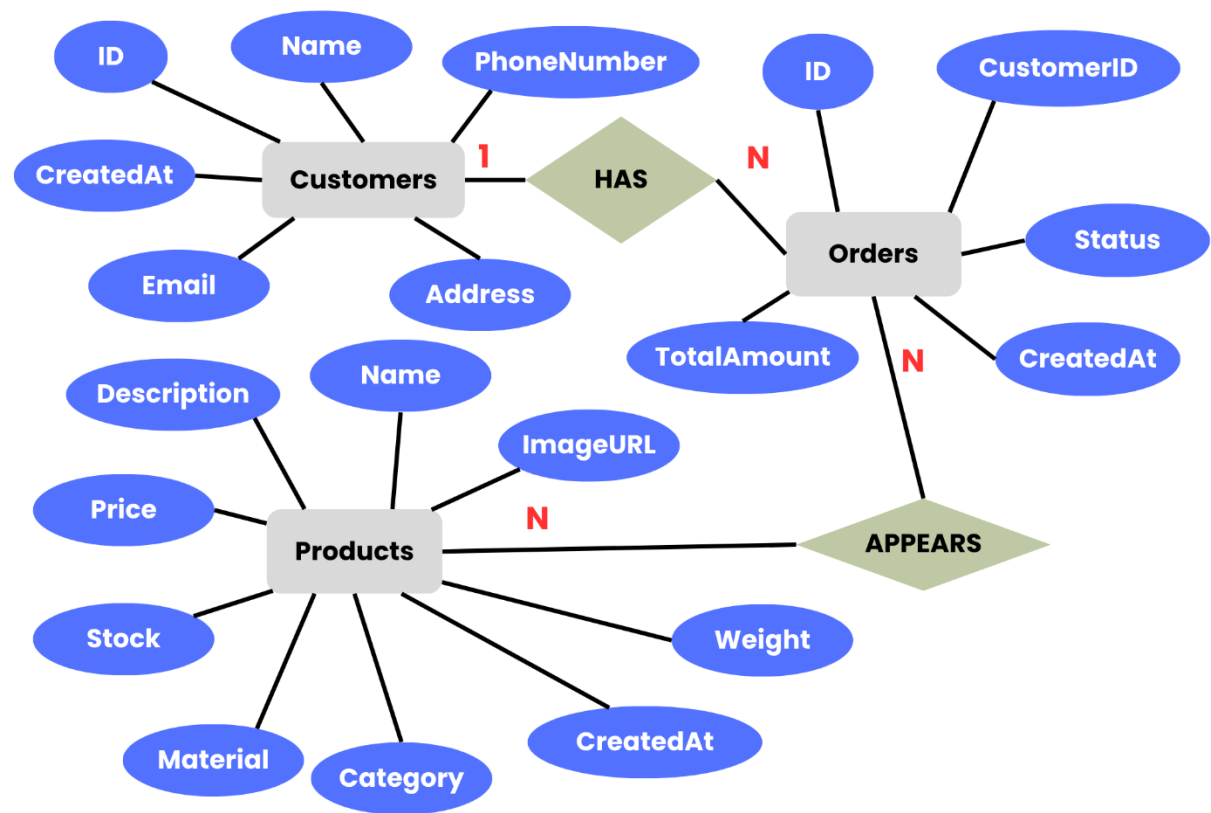
Một khách hàng có thể có nhiều đơn hàng.

- *Orders* → *Products* (Nhiều – nhiều)

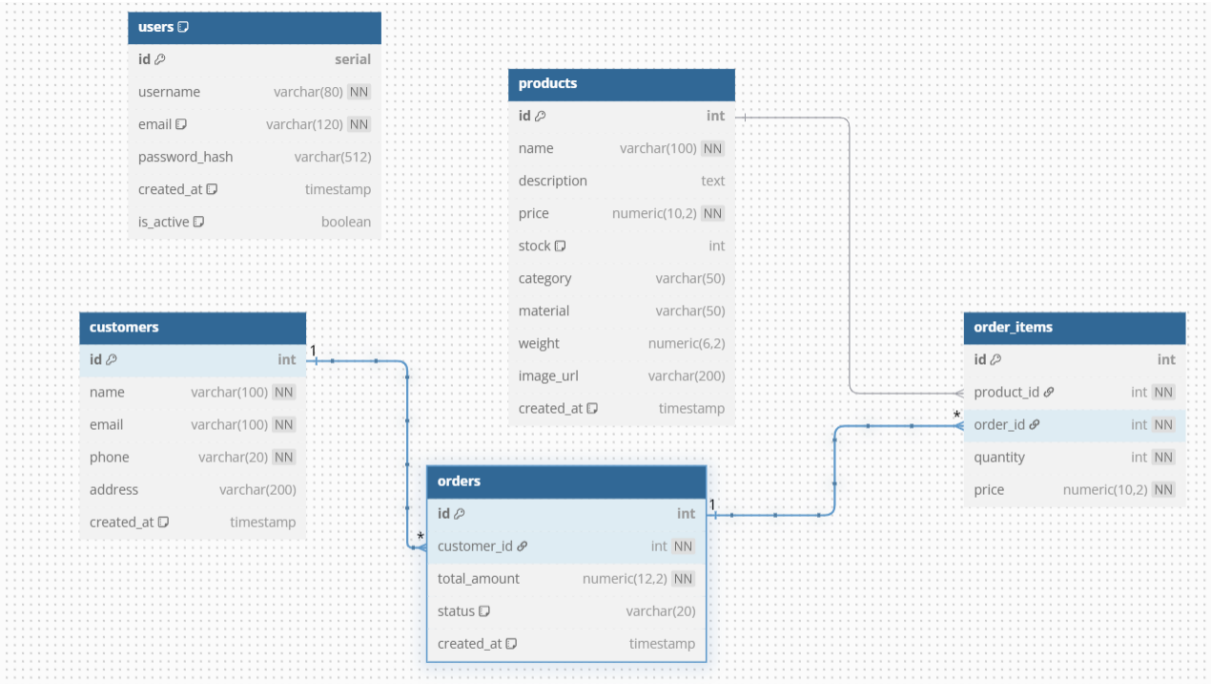
Một đơn hàng có thể có nhiều sản phẩm.

Một sản phẩm có thể xuất hiện trong nhiều đơn hàng.

C. Sơ đồ thực thể ERD



D. Sơ đồ quan hệ



IV. Tạo bảng cho cơ sở dữ liệu

A. Bảng người dùng (Users)

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(80) UNIQUE NOT NULL,  
    email VARCHAR(120) UNIQUE NOT NULL CHECK (email ~* '^[a-z  
A-Z-9._%+-]+@gmail\.com$'),  
    password_hash VARCHAR(512),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    is_active BOOLEAN DEFAULT TRUE  
);
```

B. Bảng khách hàng (Customers)

```
CREATE TABLE customers (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    phone VARCHAR(20) NOT NULL,  
    address VARCHAR(200),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Hàm trigger xử lý làm sạch và kiểm tra số điện thoại  
CREATE OR REPLACE FUNCTION clean_phone_number()  
RETURNS TRIGGER AS $$  
BEGIN  
    -- Loại bỏ tất cả ký tự không phải số  
    NEW.phone := regexp_replace(NEW.phone, '[^0-9]', '', 'g');  
  
    -- Kiểm tra đúng 10 chữ số  
    IF length(NEW.phone) <> 10 THEN  
        RAISE EXCEPTION 'Số điện thoại phải có đúng 10 chữ số  
sau khi làm sạch. Giá trị hiện tại: %', NEW.phone;  
    END IF;
```

```

        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Gắn trigger vào bảng customers
CREATE TRIGGER trg_clean_phone
BEFORE INSERT OR UPDATE ON customers
FOR EACH ROW
EXECUTE FUNCTION clean_phone_number();

```

C. Bảng sản phẩm trang sức (Products)

```

CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    price NUMERIC(10, 2) NOT NULL CHECK (price >= 0), -- Giá
không âm
    stock INTEGER DEFAULT 0 CHECK (stock >= 0), -- Tồn
kho không âm
    category VARCHAR(50),
    material VARCHAR(50),
    weight NUMERIC(6, 2) CHECK (weight IS NULL OR weight >=
0), -- Trọng lượng không âm hoặc NULL
    image_url VARCHAR(200),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

D. Bảng đơn hàng (Orders)

```

CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customers(id) ON
DELETE CASCADE,
    total_amount NUMERIC(12, 2) NOT NULL CHECK (total_amount
>= 0), -- Đảm bảo không âm

```

```

        status VARCHAR(20) DEFAULT 'pending'
        CHECK (status IN ('pending', 'processing',
'completed', 'cancelled')), -- Trạng thái hợp lệ
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

```

E. Bảng chi tiết đơn hàng (Order_Items)

```

CREATE TABLE order_items (
    id SERIAL PRIMARY KEY,
    order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE
CASCADE,
    product_id INTEGER NOT NULL REFERENCES products(id) ON
DELETE CASCADE,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
-- Ít nhất 1 sản phẩm
    price NUMERIC(10, 2) NOT NULL CHECK (price >= 0)
-- Giá không âm
);

```

V. Các chức năng nghiệp vụ

A. Quản lý sản phẩm

- **Danh sách sản phẩm sắp hết hàng**

```

CREATE OR REPLACE VIEW view_low_stock_products AS
SELECT id, name, stock
FROM products
WHERE stock < 5;

```

Test: *SELECT * FROM view_low_stock_products;*

- **Top 5 sản phẩm bán chạy nhất**

```

CREATE OR REPLACE VIEW view_best_selling_products AS
SELECT p.id, p.name, SUM(oi.quantity) AS total_sold
FROM products p JOIN order_items oi ON p.id = oi.product_id

```

```
GROUP BY p.id, p.name
ORDER BY total_sold DESC
LIMIT 5;
```

Test: SELECT * FROM view_best_selling products;

- **Tăng tốc tìm kiếm theo tên**

```
CREATE INDEX idx_product_name ON products(name);
```

- **Tìm sản phẩm bán chậm**

```
CREATE OR REPLACE FUNCTION fn_slow_selling_products(
    months INT,
    min_sales INT
)
RETURNS TABLE(
    product_id INT,
    product_name VARCHAR,
    sold_quantity INT
) AS $$
BEGIN
    RETURN QUERY
        SELECT p.id, p.name, COALESCE(SUM(oi.quantity), 0) AS
sold_quantity
        FROM products p
        LEFT JOIN order_items oi
            ON p.id = oi.product_id
            AND oi.created_at > (CURRENT_DATE - (months || '
months')::INTERVAL)
        GROUP BY p.id, p.name
        HAVING COALESCE(SUM(oi.quantity), 0) <= min_sales;
END;
$$ LANGUAGE plpgsql;
```

Test: SELECT * FROM fn_slow_selling products(3,10);

(Tìm sản phẩm bán chậm trong 3 tháng, với doanh số nhỏ hơn hoặc bằng 10)

- **Điều chỉnh giá sản phẩm dựa trên tồn kho**

```
CREATE OR REPLACE PROCEDURE sp_auto_adjust_price(  
    low_stock_threshold INT,  
    high_stock_threshold INT,  
    percent_up INT,  
    percent_down INT  
)  
LANGUAGE plpgsql AS $$  
BEGIN  
    -- Tăng giá khi tồn kho thấp hơn ngưỡng  
    UPDATE products  
    SET price = price * (1 + percent_up / 100.0)  
    WHERE stock < low_stock_threshold;  
  
    -- Giảm giá khi tồn kho cao hơn ngưỡng  
    UPDATE products  
    SET price = price * (1 - percent_down / 100.0)  
    WHERE stock > high_stock_threshold;  
END;  
$;
```

Test: CALL sp_auto_adjust_price(5, 50, 10, 5);

(Tăng giá 10% khi tồn kho dưới 5 và giảm giá 5% khi tồn kho trên 50)

- **Tự trừ stock khi thêm sản phẩm vào đơn hàng**

```
CREATE OR REPLACE FUNCTION trg_update_product_stock_func()  
RETURNS TRIGGER AS $$  
BEGIN  
    UPDATE products SET stock = stock - NEW.quantity WHERE id  
= NEW.product_id;  
    RETURN NEW;  
END;
```

```

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_product_stock
AFTER INSERT ON order_items
FOR EACH ROW
EXECUTE FUNCTION trg_update_product_stock_func();

```

- **Kiểm tra tồn kho trước khi xuất hàng**

```

CREATE OR REPLACE FUNCTION trg_check_product_stock_func()
RETURNS TRIGGER AS $$
DECLARE
    cur_stock INT;
BEGIN
    SELECT stock INTO cur_stock FROM products WHERE id =
NEW.product_id;
    IF NEW.quantity > cur_stock THEN
        RAISE EXCEPTION 'Sản phẩm không đủ số lượng trong kho!';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

DROP TRIGGER IF EXISTS trg_check_product_stock ON
order_items;
CREATE TRIGGER trg_check_product_stock
BEFORE INSERT ON order_items
FOR EACH ROW
EXECUTE FUNCTION trg_check_product_stock_func();

```

- **Khi đơn hàng bị xóa, cộng lại số lượng cho stock**

```

CREATE OR REPLACE FUNCTION
trg_restore_stock_on_order_delete()
RETURNS TRIGGER AS $$
BEGIN

```

```
-- Cộng lại số lượng cho từng sản phẩm trong đơn hàng bị  
xóa
```

```
UPDATE products  
SET stock = stock + oi.quantity  
FROM order_items oi  
WHERE oi.product_id = products.id AND oi.order_id =  
OLD.id;
```

```
RETURN OLD;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_restore_stock  
AFTER DELETE ON orders  
FOR EACH ROW  
EXECUTE FUNCTION trg_restore_stock_on_order_delete();
```

```
Test: -- Giả sử sản phẩm có id 1 và 2, tồn kho hiện tại  
-- Lưu lại tồn kho ban đầu để so sánh
```

```
WITH before_stock AS (  
    SELECT id, stock FROM products WHERE id IN (1, 2)  
) ,  
-- Thêm đơn hàng và order_items cho test  
insert_order AS (  
    INSERT INTO orders (customer_id, order_date) VALUES (999,  
CURRENT_DATE)  
    RETURNING id  
) ,  
insert_order_items AS (  
    INSERT INTO order_items (order_id, product_id, quantity)  
SELECT  
    (SELECT id FROM insert_order) ,  
    product_id ,  
    quantity  
FROM (VALUES (1, 2), (2, 3)) AS t(product_id, quantity)
```

```

),
-- Xóa đơn hàng để trigger chạy
delete_order AS (
    DELETE FROM orders WHERE id = (SELECT id FROM
insert_order)
    RETURNING *
)
SELECT
    p.id,
    p.stock,
    b.stock AS stock_before,
    p.stock - b.stock AS stock_diff
FROM
    products p
JOIN
    before_stock b ON p.id = b.id
WHERE
    p.id IN (1, 2);

```

- **Cập nhật giá hàng loạt theo phần trăm**

```

CREATE OR REPLACE PROCEDURE
sp_update_product_price(percent_increase NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE products SET price = price * (1 +
percent_increase/100);
END;
$$;

```

Test: CALL sp_update_product_price(7)
(Tăng giá tất cả sản phẩm 7%)

- **Gộp sản phẩm trùng thông tin (name, category, material, weight)**
 - **Cập nhật tổng stock**


```

WITH dupes AS (
    SELECT
        MIN(id) AS keep_id,
        ARRAY_AGG(id ORDER BY id) AS all_ids,
        SUM(stock) AS total_stock
    FROM products
    GROUP BY name, category, material, weight
    HAVING COUNT(*) > 1
)
UPDATE products p
SET stock = d.total_stock
FROM dupes d
WHERE p.id = d.keep_id;

```

◦ **Xóa bản ghi trùng**

```

DELETE FROM products
WHERE id IN (
    SELECT unnest(all_ids[2:]) -- loại bỏ phần tử đầu
    (keep_id)
    FROM (
        SELECT ARRAY_AGG(id ORDER BY id) AS all_ids
        FROM products
        GROUP BY name, category, material, weight
        HAVING COUNT(*) > 1
    ) t
);

```

Test: Cập nhật tổng stock cho bản ghi giữ lại
WITH dupes AS (

```

SELECT
MIN(id) AS keep_id,
ARRAY_AGG(id ORDER BY id) AS all_ids,
SUM(stock) AS total_stock
FROM products
GROUP BY name, category, material, weight

```

```

HAVING COUNT(*) > 1
)
UPDATE products p
SET stock = d.total_stock
FROM dupes d
WHERE p.id = d.keep_id;

-- Xóa bản ghi trùng
DELETE FROM products
WHERE id IN (
    SELECT unnest(all_ids[2:]) -- loại bỏ phần tử đầu (keep_id)
    FROM (
        SELECT ARRAY_AGG(id ORDER BY id) AS all_ids
        FROM products
        GROUP BY name, category, material, weight
        HAVING COUNT(*) > 1
    ) t
);

```

B. Quản lý đơn hàng

- 10 đơn hàng mới nhất

```

CREATE OR REPLACE VIEW view_recent_orders AS
SELECT o.id, c.name AS customer_name, o.total_amount,
o.created_at
FROM orders o JOIN customers c ON o.customer_id = c.id
ORDER BY o.created_at DESC
LIMIT 10;

```

Test: `SELECT * FROM view_recent_orders;`

- Doanh thu từng tháng

```

CREATE OR REPLACE VIEW view_monthly_revenue AS
SELECT
    DATE_TRUNC('month', created_at) AS revenue_month,
    SUM(total_amount) AS total_revenue

```

```
FROM orders
GROUP BY revenue_month
ORDER BY revenue_month DESC;
```

Test: *SELECT * FROM view_monthly_revenue;*

- **Tăng tốc truy vấn đơn hàng theo trạng thái và thời gian**

```
CREATE INDEX idx_order_status_created_at ON orders(status,
created_at);
```

- **Cảnh báo khi số đơn hàng bị hủy trong ngày vượt ngưỡng**

```
CREATE OR REPLACE PROCEDURE
sp_alert_many_cancelled_orders(threshold INT)
LANGUAGE plpgsql AS $$
DECLARE
    cancel_count INT;
BEGIN
    SELECT COUNT(*) INTO cancel_count
    FROM orders
    WHERE status = 'cancelled' AND created_at::date =
CURRENT_DATE;

    IF cancel_count >= threshold THEN
        RAISE NOTICE 'Hôm nay đã có % đơn hàng bị hủy, vui lòng
kiểm tra!', cancel_count;
        -- Thêm logic gửi email hoặc SMS cảnh báo tại đây nếu cần
    END IF;
END;
$$;
```

Test: *CALL sp_alert_many_cancelled_orders(5);*

- **Lấy tổng doanh thu**

```
CREATE OR REPLACE FUNCTION fn_total_revenue()
RETURNS NUMERIC AS $$
    SELECT COALESCE(SUM(total_amount),0) FROM orders;
$$ LANGUAGE sql;
```

Test: *SELECT fn_total_revenue();*

- **Khi tạo đơn hàng, đặt trạng thái ban đầu là ‘pending’**

```
CREATE OR REPLACE FUNCTION trg_set_order_pending_func()
RETURNS TRIGGER AS $$
BEGIN
    NEW.status := 'pending';
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_set_order_pending
BEFORE INSERT ON orders
FOR EACH ROW
EXECUTE FUNCTION trg_set_order_pending_func();
```

Test: *INSERT INTO orders(customer_id, total_amount, created_at)*
VALUES (1, 500000, NOW());

-- Kiểm tra trạng thái:

SELECT id, status FROM orders ORDER BY created_at DESC LIMIT 1;

- **Khi xóa đơn hàng, lưu thông tin vào bảng order_history**

```
CREATE TABLE IF NOT EXISTS order_history (
    id SERIAL PRIMARY KEY,
    order_id INT,
    customer_id INT,
    total_amount NUMERIC(12,2),
    status VARCHAR(20),
    deleted_at TIMESTAMP
);
```

```

CREATE OR REPLACE FUNCTION trg_log_order_delete_func()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO order_history(order_id, customer_id,
total_amount, status, deleted_at)
    VALUES (OLD.id, OLD.customer_id, OLD.total_amount,
OLD.status, now());
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

```

```

DROP TRIGGER IF EXISTS trg_log_order_delete ON orders;
CREATE TRIGGER trg_log_order_delete
AFTER DELETE ON orders
FOR EACH ROW
EXECUTE FUNCTION trg_log_order_delete_func();

```

Test: DELETE FROM orders WHERE id = 1;

-- Kiểm tra trong bảng order_history

SELECT * FROM order_history WHERE order_id = 1;

- **Cập nhật tổng tiền nếu đơn hàng có thay đổi**

```

CREATE OR REPLACE FUNCTION update_order_total_amount()
RETURNS TRIGGER AS $$
BEGIN
    -- Cập nhật tổng tiền đơn hàng dựa trên order_id
    UPDATE orders
    SET total_amount = (
        SELECT COALESCE(SUM(quantity * price), 0)
        FROM order_items
        WHERE order_id =
            CASE
                WHEN TG_OP = 'DELETE' THEN OLD.order_id
                ELSE NEW.order_id
            END
    );
END;

```

```

        END
    )
    WHERE id =
        CASE
            WHEN TG_OP = 'DELETE' THEN OLD.order_id
            ELSE NEW.order_id
        END;

    RETURN NULL; -- AFTER trigger không cần trả về NEW/OLD
END;
$$ LANGUAGE plpgsql;

-- Trigger sau INSERT
CREATE TRIGGER trg_update_order_total_after_insert
AFTER INSERT ON order_items
FOR EACH ROW
EXECUTE FUNCTION update_order_total_amount();

-- Trigger sau UPDATE
CREATE TRIGGER trg_update_order_total_after_update
AFTER UPDATE ON order_items
FOR EACH ROW
EXECUTE FUNCTION update_order_total_amount();

-- Trigger sau DELETE
CREATE TRIGGER trg_update_order_total_after_delete
AFTER DELETE ON order_items
FOR EACH ROW
EXECUTE FUNCTION update_order_total_amount();

```

C. Quản lý khách hàng

- **Top 5 khách hàng mua nhiều nhất**

```

CREATE OR REPLACE VIEW view_top_customers AS
SELECT c.id, c.name, COUNT(o.id) AS order_count,
SUM(o.total_amount) AS total_spent

```

```
FROM customers c JOIN orders o ON c.id = o.customer_id
GROUP BY c.id, c.name
ORDER BY total_spent DESC
LIMIT 5;
```

Test: *SELECT * FROM view_top_customers;*

- **Tăng tốc tìm kiếm khách hàng theo email**

```
CREATE INDEX idx_customer_email ON customers(email);
```

- **Tổng số khách hàng**

```
CREATE OR REPLACE FUNCTION fn_count_customers()
RETURNS INTEGER AS $$
    SELECT COUNT(*) FROM customers;
$$ LANGUAGE sql;
```

Test: *SELECT fn_count_customers();*

- **Lấy số đơn hàng của một khách hàng**

```
CREATE OR REPLACE FUNCTION fn_get_customer_orders(cid INT)
RETURNS INTEGER AS $$
    SELECT COUNT(*) FROM orders WHERE customer_id = cid;
$$ LANGUAGE sql;
```

Test: *SELECT fn_get_customer_orders(1)*

(Khách hàng có id = 1)

- **Đề xuất nhóm khách hàng tiềm năng dựa trên số đơn hàng và tổng chi tiêu**

```
CREATE OR REPLACE FUNCTION fn_potential_marketing_targets(
    min_orders INT,
    min_total_spent NUMERIC
)
RETURNS TABLE (
    customer_id INT,
    customer_name VARCHAR,
```

```

        total_orders INT,
        total_spent NUMERIC
    ) AS $$
BEGIN
    RETURN QUERY
    SELECT c.id, c.name, COUNT(o.id), SUM(o.total_amount)
    FROM customers c
    JOIN orders o ON c.id = o.customer_id
    GROUP BY c.id, c.name
    HAVING COUNT(o.id) >= min_orders AND SUM(o.total_amount)
    >= min_total_spent;
END;
$$ LANGUAGE plpgsql;

```

Test: SELECT * FROM fn_potential_marketing_targets(3, 1000000);

(Khách hàng có >= 3 đơn và chi tiêu >= 1000000)

- **Nâng hạng khách dựa trên tổng chi tiêu**

```

CREATE OR REPLACE FUNCTION trg_auto_upgrade_customer_level()
RETURNS TRIGGER AS $$
DECLARE
    total_spent NUMERIC;
    new_status VARCHAR;
BEGIN
    SELECT SUM(total_amount) INTO total_spent
    FROM orders
    WHERE customer_id = NEW.customer_id;

    IF total_spent >= 10000000 THEN
        new_status := 'VIP';
    ELSIF total_spent >= 3000000 THEN
        new_status := 'Thân thiết';
    ELSE
        new_status := 'Thường';
    END IF;

```



```

        UPDATE customers SET status = new_status WHERE id =
NEW.customer_id;

        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Tạo trigger kích hoạt sau khi thêm đơn hàng mới
DROP TRIGGER IF EXISTS trg_auto_upgrade_customer_level ON orders;

CREATE TRIGGER trg_auto_upgrade_customer_level
AFTER INSERT ON orders
FOR EACH ROW
EXECUTE FUNCTION trg_auto_upgrade_customer_level();

```

- **Thống kê đơn hàng của khách (tổng tiền, số đơn, trung bình đơn)**

```

CREATE OR REPLACE FUNCTION fn_get_customer_order_stats(cid
INT)
RETURNS TABLE(total_money NUMERIC, order_count INT,
avg_order NUMERIC) AS $$
    SELECT COALESCE(SUM(total_amount),0), COUNT(*),
COALESCE(AVG(total_amount),0)
    FROM orders WHERE customer_id = cid;
$$ LANGUAGE sql;

```

Test: *SELECT * FROM fn_get_customer_order_stats(1);*
(Khách hàng có id = 1)

- **Tìm khách hàng không đặt đơn trong X tháng**

```

CREATE OR REPLACE FUNCTION fn_find_inactive_customers(months
INT)
RETURNS TABLE(id INT, name VARCHAR, email VARCHAR) AS $$
    SELECT c.id, c.name, c.email
    FROM customers c
    LEFT JOIN orders o ON c.id = o.customer_id
    GROUP BY c.id, c.name, c.email

```

```
HAVING MAX(COALESCE(o.created_at, '1900-01-01')) < NOW() -  
(months || ' months')::INTERVAL;  
$$ LANGUAGE sql;
```

***Test: SELECT * FROM fn_find_inactive_customers(6);
(Tìm khách không mua trong 6 tháng)***